

Dok

Applicazioni e Servizi Web

Lorenzo Tosi, Alessandro Stefani

16 giugno 2026

Indice

1	Introduzione	3
2	Analisi dei Requisiti	4
2.1	Requisiti Funzionali	4
2.2	Requisiti Non Funzionali	5
3	Design e Architettura	6
3.1	Architettura di Sistema	6
3.2	Modello dei Dati e Sincronizzazione (CRDT)	6
3.3	Design del Backend e del Frontend	7
3.4	Progettazione UI/UX e Mockup	7
3.4.1	Dashboard e File System Virtuale	7
3.4.2	L'Editor Collaborativo	8
3.4.3	Pannello di Condivisione e Assistente IA	9
3.5	Target User Analysis	9
3.5.1	Personas e Scenari d'Uso	9
4	Tecnologie Utilizzate	11
4.1	Lo Stack MEVN come base architetturale	11
4.2	Sviluppo Frontend: Vue 3 e State Management	11
4.2.1	Gestione dello Stato con Pinia	11
4.3	Il Cuore Collaborativo: Tiptap e Yjs	12
4.4	Sviluppo Backend: Node.js ed Express.js	12
4.4.1	Integrazione con Modelli LLM	12
4.5	Comunicazione Real-Time: Socket.io	12
4.6	Persistenza dei Dati e Deployment	13
5	Implementazione e Codice	14
5.1	Gestione dell'Autenticazione e Sicurezza	14
5.1.1	L'approccio lato Client (Axios Interceptors)	15
5.2	Il Core Collaborativo: Yjs e Vue 3	15
5.3	Gestione dei Socket e Spazio Globale	16
5.4	Integrazione del Motore LLM	16
6	Test e Usabilità	18
6.1	Fase di Testing Tecnico	18
6.2	Valutazione dell'Usabilità (Euristiche di Nielsen)	18

7	Deployment	20
7.1	Struttura dei Container	20
7.2	Messa in funzione	20
8	Conclusioni	21
8.1	Considerazioni e Sfide Affrontate	21
8.2	Sviluppi Futuri	21

Capitolo 1

Introduzione

Dok mette a disposizione un ambiente di lavoro collaborativo e unisce in una singola piattaforma le funzionalità di archiviazione in cloud e l'editing simultaneo di documenti testuali.

L'applicazione si articola attorno a due macro-ambienti di interazione. Il primo è lo spazio privato: ogni utente registrato gode di una propria area di lavoro esclusiva e privata in cui può generare documenti, organizzarli in un sistema di cartelle e gestirne i permessi. Il sistema permette al proprietario del file di invitare altri utenti assegnando loro specifici ruoli: visualizzazione o modifica.

Dall'altro lato, la piattaforma integra un'area definita "globale". Quest'ultima si comporta come una bacheca pubblica dove i documenti e le cartelle in essa contenuti sono visibili liberamente a tutti, inclusi gli utenti che navigano il sito in modalità ospite senza aver effettuato l'accesso. Pur essendo le risorse pubbliche accessibili in lettura a chiunque, la piattaforma garantisce la sicurezza dei contenuti permettendo l'effettiva modifica del testo esclusivamente agli utenti autorizzati.

Capitolo 2

Analisi dei Requisiti

La fase iniziale dello sviluppo della piattaforma Dok è stata interamente dedicata alla definizione accurata dei requisiti di sistema. L'obiettivo primario consisteva nel delineare con chiarezza le necessità del target di riferimento, ponendo l'utente finale al centro di ogni scelta progettuale. Per evitare la strutturazione di funzionalità astratte, la progettazione si è basata sulle metodologie dello *User-Centered Design* (UCD), avvalendosi in particolar modo della tecnica delle *Personas* e degli scenari d'uso. Questo approccio ha permesso di simulare contesti reali di interazione, traducendoli successivamente in requisiti tecnici concreti.

2.1 Requisiti Funzionali

I requisiti funzionali descrivono in modo dettagliato le azioni che il sistema deve consentire di compiere, definendo il comportamento dell'applicazione in risposta agli input degli utenti.

- **Gestione dell'Autenticazione e Profilazione:**

- Il sistema deve permettere la registrazione di un nuovo utente tramite inserimento di username, email e password.
- L'applicazione deve consentire il login sicuro tramite emissione di un token JWT (JSON Web Token).
- Devono essere supportati tre ruoli distinti con privilegi crescenti: Ospite (non autenticato), Utente (autenticato) e Amministratore.

- **Gestione dello Spazio Privato (File System Virtuale):**

- Ogni utente autenticato deve disporre di una radice privata esclusiva.
- Il sistema deve abilitare le operazioni CRUD (creazione, lettura, rinomina, eliminazione) su cartelle e documenti organizzati ad albero.

- **Condivisione Granulare delle Risorse:**

- Il proprietario di un documento privato deve poterlo condividere con altri utenti registrati specificando l'indirizzo email del destinatario.
- Il sistema deve supportare tre livelli di permessi per le condivisioni: *Viewer* (sola lettura) ed *Editor* (modifica) e *commenter* (sola lettura con possibilità di aggiungere commenti).

- **Editor Collaborativo Real-Time:**
 - Gli utenti con permessi di scrittura sullo stesso file devono poter digitare testo simultaneamente.
 - Il sistema deve propagare istantaneamente le modifiche e mostrare i cursori attivi con i nomi degli utenti connessi.
- **Spazio Globale Pubblico:**
 - La piattaforma deve esporre una sezione "Globale" visibile anche agli utenti non autenticati.
 - Qualsiasi utente autenticato può creare elementi nell'area globale, ma la modifica collaborativa del testo rimane vincolata ai permessi specifici di scrittura.
- **Integrazione dell'Assistente IA:**
 - L'editor deve includere un menu contestuale attivabile sulla selezione del testo che consenta di richiedere la riscrittura, la sintesi o il perfezionamento del frammento selezionato tramite un motore LLM integrato nel backend.
- **Monitoraggio e Amministrazione:**
 - L'utente con ruolo di Amministratore deve poter accedere a una dashboard riservata per visualizzare l'elenco degli utenti, ispezionare l'intero file system globale e monitorare i log di sistema.
- **Notifiche**
 - La piattaforma deve avere un sistema di notifica che aggiorna l'utente quando gli viene condiviso un nuovo documento, quando gli vengono aggiornati i permessi e quando gli vengono rimossi.

2.2 Requisiti Non Funzionali

I requisiti non funzionali specificano le caratteristiche qualitative, i vincoli tecnici e le metriche di efficienza che il software deve rispettare durante il suo funzionamento.

- **Usabilità e Approccio Mobile First:** L'interfaccia grafica deve essere flessibile. Il layout deve adattarsi automaticamente alle dimensioni dello schermo del dispositivo utilizzato, garantendo un'esperienza fluida sia su desktop che su smartphone e tablet.
- **Reattività e Sincronizzazione a Bassa Latenza:** Il flusso dei dati collaborativi deve essere immediato. La propagazione degli aggiornamenti testuali tra i client connessi allo stesso documento deve avvenire in tempo reale, mantenendo la latenza impercettibile per l'occhio umano.
- **Consistenza dei Dati e Risoluzione dei Conflitti:** Il sistema non deve generare incongruenze. In caso di modifiche simultanee sullo stesso paragrafo, l'applicazione deve integrare i testi in modo matematicamente consistente senza causare sovrascritture distruttive o disallineamenti tra i client.
- **Sicurezza e Architettura Stateless:** La protezione delle risorse è un vincolo stringente. Tutte le rotte API riservate e le connessioni WebSocket devono essere protette da meccanismi di autorizzazione basati su token crittografici, evitando di memorizzare lo stato della sessione sulla memoria del server backend.

Capitolo 3

Design e Architettura

Per tradurre in realtà i requisiti definiti nella fase di analisi, la progettazione del sistema Dok ha richiesto un'attenta pianificazione dell'infrastruttura software. La natura fortemente interattiva e collaborativa dell'applicazione ha imposto l'adozione di pattern architetturali in grado di gestire non solo le tradizionali operazioni CRUD, ma anche un flusso di dati continuo e bidirezionale tra i client e il server.

3.1 Architettura di Sistema

L'applicativo si fonda sul paradigma Client-Server, strutturato per garantire una netta separazione delle responsabilità (Separation of Concerns). Questa scelta permette di mantenere il frontend leggero e puramente dedicato alla presentazione e alla reattività dell'interfaccia, delegando al backend tutta la complessa logica di validazione, autenticazione e persistenza dei dati.

Per gestire la diversità delle operazioni richieste, il sistema utilizza due modelli di comunicazione:

- **Architettura RESTful:** Utilizzata per tutte le operazioni di natura statica e transazionale. Tramite chiamate HTTP standard, il client comunica con le API del server per gestire l'autenticazione, recuperare l'albero delle cartelle, creare nuovi documenti o aggiornare il profilo utente.
- **Comunicazione Event-Driven (WebSocket):** Fondamentale per il cuore collaborativo dell'app. Attraverso una connessione persistente bidirezionale, il server effettua la *push* delle notifiche, degli aggiornamenti di stato della dashboard e, soprattutto, delle modifiche testuali in tempo reale, senza che il client debba continuamente interrogare il server (polling).

Figura 3.1: Diagramma dell'architettura Client-Server e dei flussi di comunicazione.

3.2 Modello dei Dati e Sincronizzazione (CRDT)

La sfida architetturale più complessa consisteva nel garantire la consistenza del testo modificato simultaneamente da più utenti. Se due persone scrivono contemporaneamente sullo stesso paragrafo, l'uso di semplici lock o sovrascritture asincrone porterebbe a perdite di dati e a una pessima esperienza utente.

Per risolvere in modo elegante questo problema, l'architettura integra il modello dei **CRDT** (*Conflict-free Replicated Data Types*). Piuttosto che delegare al server l'onere di unire le versioni del documento (come avviene nei tradizionali sistemi di versioning), la logica di risoluzione dei conflitti è distribuita. Ogni utente mantiene una copia locale della struttura dati del documento. Quando un utente digita una lettera, il framework genera un aggiornamento incrementale che viene inviato al backend tramite WebSocket. Il server agisce da semplice *router*, inoltrando l'aggiornamento agli altri client connessi alla medesima "stanza" (room). I client ricevuti applicano deterministicamente l'aggiornamento alla propria copia locale, convergendo sempre verso il medesimo stato finale senza bisogno di bloccare l'editor.

3.3 Design del Backend e del Frontend

Il server è stato progettato seguendo un pattern modulare basato su *Controller* e *Services*. I Controller fungono da punto di ingresso per le rotte API, occupandosi dell'estrazione dei parametri e dell'invio delle risposte HTTP. La business logic vera e propria risiede invece nei Services, i quali astraggono l'interazione con il database non relazionale (orientato ai documenti), garantendo codice pulito, testabile e riutilizzabile.

Lato frontend, si è adottato un approccio *Component-Based*. L'interfaccia è composta da "mattoncini" visivi incapsulati e riutilizzabili. Per evitare il noto problema del *prop-drilling* (il passaggio di dati in cascata attraverso decine di componenti annidati), lo stato globale dell'applicazione è gestito in modo centralizzato tramite specifici *Store*. Questi contenitori reattivi tengono traccia della sessione dell'utente, dei documenti correntemente aperti e delle notifiche in sospenso, aggiornando automaticamente la vista ogni qualvolta i dati sottostanti mutano.

3.4 Progettazione UI/UX e Mockup

L'interfaccia utente è stata concepita seguendo i principi del *KISS* (Keep It Simple, Stupid) e del *Minimalist Design*. L'intento era quello di ridurre al minimo il carico cognitivo dell'utente, fornendo un'esperienza visiva familiare che richiamasse i software di produttività cloud più diffusi.

La progettazione è partita con la realizzazione di mockup ad alta fedeltà, assicurandosi che le interfacce fossero pienamente responsive e godibili anche da dispositivi mobili.

3.4.1 Dashboard e File System Virtuale

La vista principale dopo l'accesso è la Dashboard. L'interfaccia si divide in una comoda barra laterale per la navigazione rapida (con collegamenti allo Spazio Privato e all'Area Globale) e in un'ampia griglia centrale che mostra le cartelle e i documenti sotto forma di schede visuali.

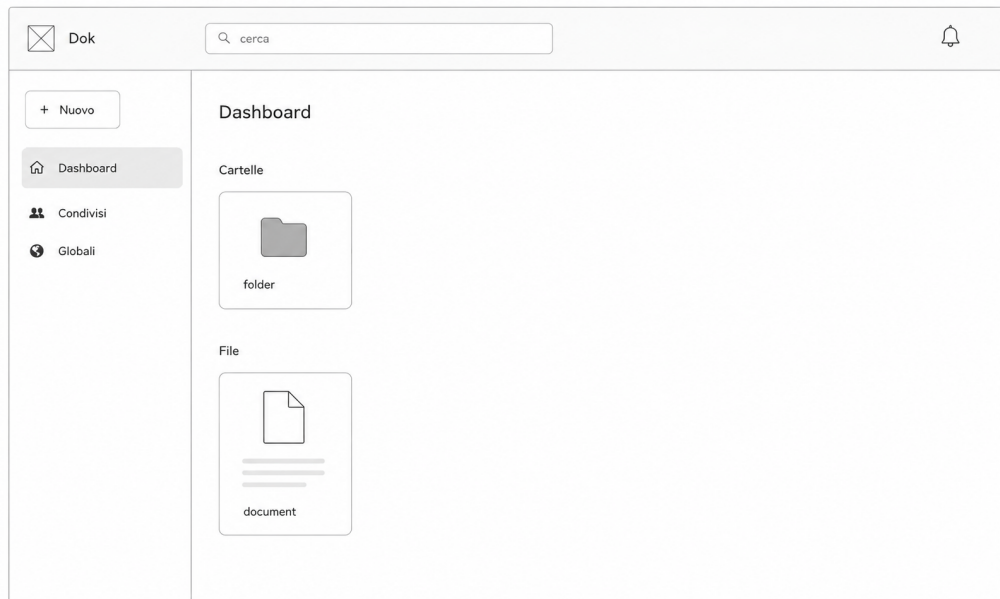


Figura 3.2: Mockup della Dashboard principale.

3.4.2 L'Editor Collaborativo

L'editor si presenta essenziale e privo di distrazioni. Nella parte superiore, una barra degli strumenti racchiude le opzioni di formattazione. È presente un indicatore delle presenze: avatar colorati mostrano in tempo reale chi è connesso al documento. Questa scelta di design non è puramente estetica, ma funge da *awareness protocol*, assicurando l'utente sul fatto che la sessione collaborativa è attiva.

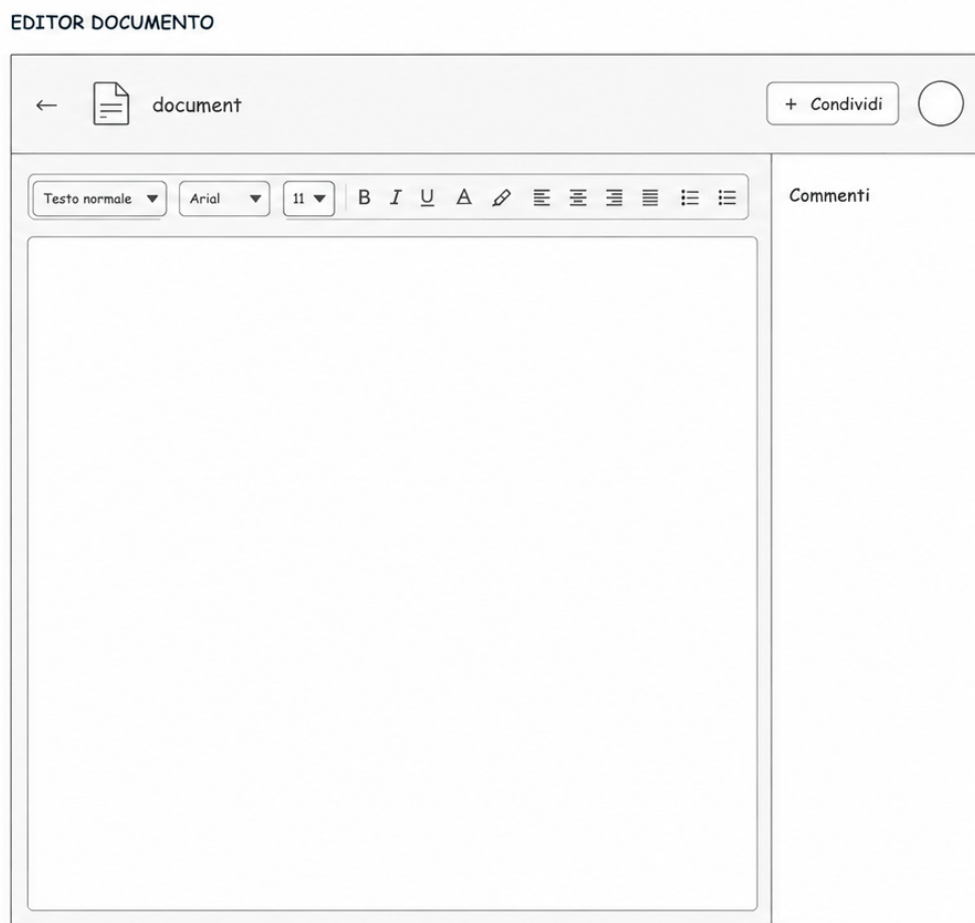


Figura 3.3: Mockup dell’editor dei documenti.

3.4.3 Pannello di Condivisione e Assistente IA

Particolare attenzione è stata data ai menu contestuali. La finestra modale per la condivisione risulta chiara, permettendo di inserire un’email e selezionare rapidamente il ruolo (Viewer o Editor) da un menu a tendina. Similmente, evidenziando una porzione di testo all’interno dell’editor, un *bubble menu* flottante compare vicino al cursore, suggerendo le opzioni di interazione con l’Intelligenza Artificiale per la riscrittura o l’espansione dei paragrafi.

3.5 Target User Analysis

Per mappare le differenti modalità di interazione con la piattaforma, sono stati individuati tre profili di utenti tipo. Ciascun profilo rappresenta una specifica fetta di pubblico, caratterizzata da esigenze, competenze digitali e obiettivi differenziati.

3.5.1 Personas e Scenari d’Uso

Persona 1: Marco (Lo studente universitario)

Marco ha 22 anni ed è uno studente iscritto alla facoltà di Informatica. Si trova spesso a dover gestire progetti di gruppo complessi, che richiedono la stesura a più mani di

relazioni tecniche, report di laboratorio e appunti delle lezioni. Ha una forte familiarità con gli strumenti digitali, ma soffre per la frammentazione dei file scambiati continuamente tramite chat.

Scenario d'uso: Marco accede a Dok per avviare la stesura della relazione finale di un esame. Nella sua area privata, crea una cartella dedicata al corso e al suo interno crea un nuovo documento di testo. Successivamente, utilizza la funzionalità di condivisione inserendo le email dei suoi due compagni di gruppo ed assegnando loro i permessi di *Editor*. I tre studenti si collegano contemporaneamente la sera stessa: ognuno scrive una sezione diversa del documento, visualizzando in tempo reale la posizione dei cursori altrui e le modifiche apportate, completando il lavoro in poche ore senza alcun conflitto di versione.

Persona 2: Giulia (La divulgatrice digitale)

Giulia ha 28 anni, lavora come content creator indipendente e cura un blog incentrato sulla tecnologia e sul self-improvement. Ha la necessità di pubblicare articoli, guide e raccoglitori di risorse testuali che siano immediatamente accessibili alla sua community, senza che i lettori debbano necessariamente registrarsi o effettuare il login per consultare i materiali.

Scenario d'uso: Giulia utilizza lo *Spazio Globale* di Dok. Questa sezione pubblica le consente di organizzare il proprio piano editoriale creando cartelle tematiche visibili a chiunque navighi sulla piattaforma. Quando redige una nuova guida, lavora nel suo spazio privato condividendo il file in sola lettura con i suoi collaboratori più stretti per ricevere un feedback visivo. Una volta rifinito il testo, sposta il documento definitivo nell'area globale, rendendolo istantaneamente accessibile in modalità consultazione a migliaia di visitatori anonimi.

Persona 3: Alessandro (Il visitatore occasionale)

Alessandro ha 34 anni ed è un utente che naviga sul web alla ricerca di documentazione tecnica e template pronti all'uso. Non ha intenzione di creare account su ogni sito che visita e apprezza l'immediatezza nell'accesso alle informazioni.

Scenario d'uso: Alessandro trova sui social un link che rimanda a una cartella condivisa nello Spazio Globale di Dok. Cliccando sul collegamento, accede alla piattaforma direttamente in modalità ospite. L'interfaccia gli mostra l'albero delle cartelle pubbliche e gli permette di aprire i documenti testuali e leggerne il contenuto. Non potendo modificare il testo, Alessandro apprezza la pulizia dell'editor in sola lettura e la rapidità con cui ha ottenuto le informazioni cercate senza dover compilare alcun modulo di registrazione.

Capitolo 4

Tecnologie Utilizzate

La piattaforma Dok è stata costruita adottando lo stack **MEVN** (MongoDB, Express.js, Vue.js, Node.js) assieme ad un insieme di librerie specializzate per risolvere problemi complessi come la convergenza dei testi e la gestione dello stato distribuito.

4.1 Lo Stack MEVN come base architetturale

La scelta dello stack MEVN si basa sul paradigma *JavaScript everywhere*. Utilizzare un unico linguaggio di programmazione sia per il client che per il server riduce la difficoltà nello sviluppo, agevolando la condivisione di interfacce (tipizzate grazie a TypeScript) e formati di dati (JSON) tra i vari layer dell'applicazione. Questa uniformità ha permesso di accelerare lo sviluppo e minimizzare gli errori di traduzione dei dati tra frontend e database.

4.2 Sviluppo Frontend: Vue 3 e State Management

L'interfaccia utente è stata interamente sviluppata utilizzando **Vue.js** (nella sua versione 3), sfruttando le potenzialità della *Composition API* e della sintassi `<script setup>`. Questo approccio ha permesso di scrivere componenti più compatti e logiche facilmente riutilizzabili sotto forma di *composables*. Il building e l'ottimizzazione degli asset sono stati affidati a **Vite**, un bundler moderno che garantisce tempi di ricarica istantanei in fase di sviluppo (Hot Module Replacement), migliorando la Developer Experience rispetto ai classici configuratori basati su Webpack.

4.2.1 Gestione dello Stato con Pinia

Data la mole di informazioni dinamiche da visualizzare (albero dei file, permessi correnti, token di autenticazione e notifiche in tempo reale), gestire lo stato tramite il semplice passaggio di *props* sarebbe risultato inefficiente e prone a bug. Per ovviare a questo problema, l'applicazione si affida a **Pinia**, lo standard moderno per lo *state management* nell'ecosistema Vue. Sono stati creati *store* modulari indipendenti (ad esempio per l'autenticazione, la gestione delle cartelle e dei documenti). Pinia garantisce che, ogni volta che un dato viene aggiornato in background (magari a seguito di una notifica WebSocket), tutti i componenti dell'interfaccia che dipendono da quel frammento di stato reagiscano e si renderizzino nuovamente in modo automatico.

4.3 Il Cuore Collaborativo: Tiptap e Yjs

La vera sfida del progetto consisteva nell'implementazione dell'editor condiviso. Una classica casella di testo o un normale editor WYSIWYG (What You See Is What You Get) non possiede nativamente gli strumenti matematici per fondere le modifiche provenienti da utenti multipli, generando inevitabili sovrascritture.

Per costruire questa funzionalità si è optato per:

- **Tiptap:** È un framework per editor di testo "headless", ovvero privo di un'interfaccia grafica predefinita. Tiptap fornisce solo le API e le logiche di rendering del testo, lasciando al programmatore il controllo totale sul design. Questo ha permesso di integrare menù flottanti personalizzati (come il *Bubble Menu* per l'assistente IA) e di mantenere uno stile estetico perfettamente coerente con il resto dell'applicazione.
- **Yjs (Implementazione CRDT):** Yjs è il vero "cervello" matematico dell'editor. Si tratta di una libreria ad alte prestazioni basata sui *Conflict-free Replicated Data Types* (CRDT). Invece di far transitare interi blocchi di testo sulla rete, Yjs scompone il documento in una struttura dati vettoriale. Quando un utente digita, la libreria calcola un *delta* infinitesimale e lo propaga. Yjs garantisce matematicamente che, indipendentemente dall'ordine in cui i pacchetti di rete arrivano ai vari client, tutti convergeranno sempre verso lo stesso identico stato finale, azzerando i conflitti.

4.4 Sviluppo Backend: Node.js ed Express.js

Il server dell'applicazione è alimentato da **Node.js**, ideale per gestire un elevato numero di connessioni I/O simultanee (come quelle richieste dalle sessioni collaborative). L'infrastruttura di routing e la creazione delle API RESTful poggiano su **Express.js**. Il codice del backend è stato ingegnerizzato facendo largo uso di *middleware*. Un esempio è la pipeline di sicurezza: prima di processare qualsiasi richiesta sensibile, il sistema intercetta la chiamata, estrae il token JWT dalle intestazioni HTTP e ne verifica la firma crittografica. Solo se l'esito è positivo, la richiesta viene inoltrata ai *Controller* e, successivamente, ai *Services* che incapsulano la logica di business.

4.4.1 Integrazione con Modelli LLM

Per arricchire l'esperienza utente, il server ospita anche un motore per l'interazione con l'Intelligenza Artificiale (LLM Engine). Questo modulo agisce come un ponte: riceve la porzione di testo selezionata dall'utente nel frontend e la instrada verso un Large Language Model, applicando specifici *prompt* per richiedere la riscrittura, l'espansione o il riassunto del paragrafo. La risposta viene poi ripulita e restituita al client, pronta per essere iniettata nel documento tramite Tiptap.

4.5 Comunicazione Real-Time: Socket.io

Per eludere i limiti del protocollo HTTP (che è nativamente unidirezionale e stateless), l'architettura sfrutta il protocollo WebSocket tramite la libreria **Socket.io**. Socket.io non si limita a tenere aperto un canale bidirezionale, ma fornisce astrazioni cruciali per la scalabilità del progetto. Nello specifico, si è fatto un largo uso delle *Rooms* (stanze). Quando tre utenti aprono il medesimo documento, il backend li iscrive automaticamente alla stessa Room. In questo modo, gli aggiornamenti di Yjs e la trasmissione della posizione dei

cursori (*Awareness Protocol*) vengono inviati in multicast (broadcast selettivo) esclusivamente agli utenti interessati da quello specifico file, ottimizzando il consumo di banda e la reattività del server.

4.6 Persistenza dei Dati e Deployment

La persistenza delle informazioni strutturate è stata affidata a **MongoDB**, un database NoSQL orientato ai documenti. La sua flessibilità nativa (schema-less) si sposa alla perfezione con l'eterogeneità dei dati testuali. L'interazione con il database è gestita tramite **Mongoose**, un Object Data Modeling (ODM) che permette di definire schemi rigorosi e validazioni direttamente nel codice TypeScript prima del salvataggio fisico dei dati.

Ad affiancare MongoDB, è stata integrata un'istanza **Redis**. Operando interamente in memoria (in-memory data structure store), Redis viene impiegato per alleggerire il carico del database primario e velocizzare le operazioni ad alta frequenza, fungendo da sistema di caching e supporto per la scalabilità orizzontale delle sessioni Socket.io.

Infine, l'intera architettura è stata inserita in container tramite **Docker**. La configurazione in container isolati garantisce che il database, il server e il frontend girino sempre in un ambiente immutabile, annullando il rischio di malfunzionamenti legati alle dipendenze o alla configurazione del sistema operativo sottostante.

Capitolo 5

Implementazione e Codice

In questo capitolo verranno analizzati i frammenti di codice più significativi del progetto, evidenziando le scelte implementative adottate per risolvere problematiche specifiche, quali la protezione degli endpoint, la gestione dello stato reattivo sul client e, soprattutto, la sincronizzazione collaborativa in tempo reale.

5.1 Gestione dell'Autenticazione e Sicurezza

L'applicazione gestisce dati sensibili e documenti privati. Di conseguenza, il sistema di autenticazione non poteva limitarsi a un semplice controllo a livello di interfaccia, ma doveva essere radicato profondamente nel backend. Si è scelto di utilizzare i JSON Web Token (JWT) per instaurare una comunicazione *stateless*.

Nel server (basato su Express), ogni rotta che espone o manipola risorse protette è filtrata da un middleware custom. Il file `auth.middleware.ts` si occupa di intercettare la richiesta HTTP, estrarre il token dall'intestazione `Authorization` e validarne la firma crittografica.

Listing 5.1: Implementazione del middleware di autenticazione (`auth.middleware.ts`)

```
import { Request, Response, NextFunction } from 'express';
import jwt from 'jsonwebtoken';

export const requireAuth = (req: Request, res: Response, next:
  NextFunction) => {
  // Estrazione del token dall'header
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Accesso negato: Token
      mancante' });
  }

  const token = authHeader.split(' ')[1];

  try {
    // Verifica e decodifica del payload
    const decoded = jwt.verify(token, process.env.JWT_SECRET as
      string);

    // Iniezione dei dati utente nell'oggetto Request
    req.user = decoded;
    next(); // Passaggio del controllo al Controller successivo
  }
```

```

    } catch (err) {
      return res.status(403).json({ error: 'Token non valido o
        scaduto' });
    }
  };

```

Se il token è valido, il payload decodificato (contenente l'ID utente e il suo ruolo) viene iniettato direttamente nell'oggetto `req`, rendendo queste informazioni immediatamente fruibili ai controller successivi senza dover interrogare nuovamente il database.

5.1.1 L'approccio lato Client (Axios Interceptors)

Per evitare di dover allegare manualmente il token in ogni singola richiesta HTTP dal frontend verso il server, si è sfruttato il meccanismo degli *interceptors* della libreria Axios all'interno di `api.ts`. Questo strumento permette di alterare silenziosamente le richieste in uscita, aggiungendo l'header necessario. Inoltre, gestisce globalmente le risposte: se il backend restituisce un errore 401 (Unauthorized), l'interceptor ordina automaticamente al router di Vue di reindirizzare l'utente alla pagina di login, pulendo lo *store* di Pinia.

5.2 Il Core Collaborativo: Yjs e Vue 3

La sincronizzazione del testo tra più client simultanei rappresenta la complessità maggiore del progetto. L'architettura sviluppata si appoggia a Yjs e al suo concetto di *Shared Document* (Y.Doc). La logica di connessione e legame con l'editor Tiptap è stata sapientemente astratta all'interno di un composable di Vue denominato `useCollaboration.ts`.

Questo pattern sfrutta il ciclo di vita dei componenti di Vue (come `onMounted` e `onBeforeUnmount`) per instaurare e distruggere la connessione WebSocket al momento giusto, prevenendo cali di prestazioni o **memory leak**.

Listing 5.2: Configurazione dell'Editor Collaborativo (`useCollaboration.ts`)

```

import * as Y from 'yjs';
import { useEditor } from '@tiptap/vue-3';
import Collaboration from '@tiptap/extension-collaboration';
import CollaborationCursor from '@tiptap/extension-collaboration-cursor';

export function useCollaboration(documentId: string, currentUser:
any, socket: any) {
  // Creazione del documento condiviso Yjs
  const ydoc = new Y.Doc();

  // Inizializzazione del provider WebSocket per la
  // sincronizzazione
  const provider = new TiptapCollabProvider({
    name: documentId,
    document: ydoc,
    wsProvider: socket, // Utilizza l'istanza Socket.io passata
      come parametro
  });

  // Configurazione dell'istanza Tiptap
  const editor = useEditor({
    extensions: [

```



```

        StarterKit,
        Collaboration.configure({
            document: ydoc
        }),
        // Gestione dei cursori e dell'Awareness
        CollaborationCursor.configure({
            provider: provider,
            user: {
                name: currentUser.username,
                color: currentUser.color
            }
        }),
    ],
});

return { editor, provider };
}

```

Il frammento precedente mostra come l'oggetto `ydoc` diventi la "singola fonte di verità" per il testo. L'estensione `CollaborationCursor` raccoglie i movimenti del mouse e le posizioni del cursore locale, impacchettandoli e inviandoli al server. Gli altri client, ricevendo tali pacchetti dal server, aggiornano le coordinate e renderizzano il cursore colorato (indicatore di **Awareness**) con il nome dell'utente.

5.3 Gestione dei Socket e Spazio Globale

L'efficienza del sistema è garantita da una rigida gestione delle *Rooms* all'interno del modulo `document.handler.ts` sul server. Quando un utente apre un file (sia esso nello Spazio Privato o in quello Globale), il frontend emette un evento `join-document`. Il server verifica prima di tutto i permessi di lettura sul database. Se l'utente ha diritto di accesso (o se il file si trova nell'area pubblica globale), il server iscrive la sua connessione Socket alla stanza corrispondente all'ID del documento.

Da quel momento in poi, qualsiasi evento di `sync-update` inviato da un *Editor* autorizzato verrà "riflesso" dal server esclusivamente ai membri di quella specifica stanza, garantendo l'isolamento dei flussi di dati e riducendo al minimo il sovraccarico di rete.

5.4 Integrazione del Motore LLM

Un ulteriore elemento tecnico è l'assistente basato su Intelligenza Artificiale. Il flusso si attiva quando l'utente seleziona una porzione di testo ed esegue una richiesta tramite il *BubbleMenu*. Il client invoca una chiamata REST al controller `llm.controller.ts`, il quale delega la complessa interazione con il modello linguistico alla classe dedicata `LLMEngine.ts`.

Listing 5.3: Elaborazione della richiesta LLM (LLMEngine.ts)

```
export class LLMEngine {
  private openai;

  constructor() {
    this.openai = new OpenAI({ apiKey: process.env.
      OPENAI_API_KEY });
  }

  public async processText(instruction: string, text: string):
    Promise<string> {
    try {
      const response = await this.openai.chat.completions.
        create({
          model: "gpt-4",
          temperature: 0.7,
          messages: [
            {
              role: "system",
              content: "Sei un assistente editoriale
                integrato in una piattaforma di
                videoscrittura. Il tuo compito:
                riscrivere o migliorare il testo fornito
                rispettando rigorosamente le istruzioni
                dell'utente."
            },
            {
              role: "user",
              content: `Istruzione: ${instruction}\n\
                nTesto originale:\n${text}`
            }
          ]
        });
      return response.choices[0].message?.content || "";
    } catch (error) {
      console.error("Errore durante la generazione LLM:",
        error);
      throw new Error("Impossibile elaborare la richiesta in
        questo momento.");
    }
  }
}
```

Si è scelto di racchiudere questa logica in una classe apposita seguendo il principio di *Single Responsibility*. Il Controller si occupa esclusivamente di estrarre l'istruzione (ad esempio "Rendi il testo più formale") e il blocco di testo dalla richiesta HTTP. La classe `LLMEngine` incapsula invece i dettagli implementativi delle API esterne. Se in futuro si decidesse di migrare da OpenAI verso un modello *open-source* locale, basterà modificare unicamente i metodi di questa classe, senza alterare il resto della logica dell'applicazione.

Capitolo 6

Test e Usabilità

Prima di procedere con il rilascio dell'applicativo, il sistema è stato sottoposto a un'intensa fase di testing al fine di validarne la stabilità, l'affidabilità e l'effettiva fruibilità da parte degli utenti finali. Le procedure di verifica hanno riguardato sia la robustezza del codice backend sia l'esperienza d'uso dell'interfaccia frontend.

6.1 Fase di Testing Tecnico

Le funzionalità del sistema sono state verificate dal team di sviluppo attraverso diversi dispositivi e sui principali browser web (Google Chrome, Mozilla Firefox, Safari e Microsoft Edge) per garantire la massima portabilità e compatibilità cross-browser.

Le API RESTful del server sono state collaudate singolarmente utilizzando *Postman*, simulando richieste con token JWT validi, scaduti o contraffatti, per accertarsi che i middleware di sicurezza intervenissero correttamente bloccando le operazioni non autorizzate. Particolare attenzione è stata riservata ai test di concorrenza sull'editor: sono state simulate sessioni con connessioni multiple allo stesso documento (sia da desktop che da mobile) per confermare che l'infrastruttura Yjs e Socket.io gestisse senza incertezze le modifiche simultanee e le disconnessioni improvvise.

6.2 Valutazione dell'Usabilità (Euristiche di Nielsen)

Per avere un riscontro qualitativo oggettivo sull'interfaccia grafica, il sistema è stato valutato seguendo i celebri principi di usabilità definiti da Jakob Nielsen. Di seguito vengono riportate le considerazioni emerse per le euristiche più rilevanti:

- **Visibilità dello stato del sistema:** L'utente viene costantemente informato su ciò che sta accadendo. Ad esempio, quando l'editor sta salvando le modifiche o l'Intelligenza Artificiale sta elaborando una riscrittura del testo, compaiono indicatori di caricamento (spinner) o micro-notifiche di sistema.
- **Corrispondenza tra sistema e mondo reale:** La terminologia e l'iconografia impiegate (cartelle, icone del drive, pulsanti di condivisione) richiamano esplicitamente i concetti fisici e i pattern già assimilati dagli utenti tramite l'uso di altre piattaforme cloud famose.
- **Controllo e libertà:** La navigazione non è mai vincolante. Se l'utente apre per sbaglio la finestra modale di condivisione o avvia un processo indesiderato, sono

sempre presenti pulsanti di annullamento e icone di chiusura chiare, permettendo di tornare allo stato precedente senza frustrazioni.

- **Consistenza e Standard:** Si è mantenuta una stretta coerenza visiva e cromatica in tutte le pagine. I colori primari indicano le azioni principali (come il "Crea Nuovo Documento"), mentre le palette secondarie guidano l'occhio verso le informazioni di contorno.
- **Prevenzione degli errori:** Le form di registrazione e di creazione file sono dotate di validazione *client-side*. Inoltre, le azioni potenzialmente distruttive, come l'eliminazione definitiva di una cartella contenente vari documenti, richiedono sempre un'esplicita conferma tramite un dialog popup.

Capitolo 7

Deployment

Per semplificare la distribuzione del sistema e rendere l'applicativo completamente indipendente dalla macchina su cui viene eseguito, si è optato per un processo di deployment basato su **Docker**. L'architettura a container assicura che il software giri sempre in un ambiente isolato e immutabile, risolvendo alla radice il classico problema delle dipendenze mancanti o delle versioni incompatibili ("it works on my machine").

7.1 Struttura dei Container

Il progetto è orchestrato tramite **docker-compose**, che si occupa di avviare e far dialogare le diverse anime dell'applicazione all'interno di una rete virtuale interna isolata. Nello specifico, il file di configurazione definisce i seguenti servizi:

- **Database (MongoDB e Redis):** Container dedicati all'archiviazione persistente dei documenti e alla gestione della cache e delle sessioni in memoria.
- **Backend (Server Node.js):** Il container che ospita la logica di business, le API RESTful e i processi WebSocket. Comunica internamente con i container dei database e accetta connessioni sulla porta dedicata.
- **Frontend (Client Vue 3):** Il container responsabile di servire gli asset statici dell'interfaccia utente, generalmente esposto sulla porta principale per l'accesso tramite browser.

7.2 Messa in funzione

Per avviare l'intero ecosistema sul proprio ambiente locale o su un server remoto, i passaggi richiesti sono stati ridotti al minimo. Dopo aver posizionato eventuali chiavi (come il JWT Secret o l'API Key di OpenAI) in un file `.env` nella root del progetto, è sufficiente aprire il terminale ed eseguire il seguente comando:

```
# Creazione e avvio di tutti i container in background
docker-compose up --build -d
```

Una volta terminato il processo di compilazione delle immagini, la piattaforma Dok sarà immediatamente accessibile navigando all'indirizzo `http://localhost:5173` (o alla porta configurata per il client), mentre le chiamate API verranno direzionate automaticamente al container del server.

Capitolo 8

Conclusioni

Lo sviluppo della piattaforma Dok si è rivelato un’ottima esperienza formativa permettendoci di affrontare e risolvere problematiche reali legate al mondo dei sistemi distribuiti e delle architetture web moderne. L’obiettivo iniziale – creare un ambiente cloud in stile Google Drive che unisse l’archiviazione alla stesura collaborativa – è stato ampiamente raggiunto.

8.1 Considerazioni e Sfide Affrontate

Non sono mancate le difficoltà tecniche durante l’avanzamento dei lavori. La sfida principale ha riguardato sicuramente l’integrazione del motore Yjs per la risoluzione dei conflitti nell’editor testuale. Comprendere a fondo il funzionamento dei CRDT e far dialogare correttamente Tiptap con l’infrastruttura di Socket.io ha richiesto uno studio approfondito, ma il risultato finale ha ampiamente ripagato gli sforzi, garantendo un’esperienza di co-editing priva di sbavature. Allo stesso modo, la strutturazione rigorosa del backend con TypeScript e la progettazione di un file system virtuale sicuro e navigabile hanno consolidato la nostra comprensione dei pattern di sviluppo lato server.

8.2 Sviluppi Futuri

Le solide basi architetturelle poste con questo progetto permettono di immaginare diverse estensioni future per arricchire ulteriormente le funzionalità dell’applicazione:

- **Sistema di chat integrata:** Aggiungere un pannello di messaggistica laterale all’interno del documento per permettere agli utenti connessi di dialogare senza abbandonare l’editor.
- **Cronologia delle revisioni:** Implementare un sistema per salvare versioni ”congelate” del documento a intervalli regolari, garantendo la possibilità di effettuare un rollback a stadi precedenti del testo.
- **Esportazione Multiformato:** Arricchire le funzionalità di download offrendo all’utente la possibilità di esportare il documento scritto non solo in testo semplice, ma anche in formato PDF formattato o Microsoft Word (.docx).